

MEMORIA DE PRÁCTICA

UT3 – Práctica A9.1

Tests Unitarios con JUnit 5 en NetBeans

Campo	Detalle
Alumno	Juan Manuel Barbosa Camacho
Módulo	Entornos de Desarrollo
Unidad	UT3 – Pruebas del Software
Práctica	A9.1 – Tests Unitarios
Herramientas	NetBeans IDE + JUnit 5
Fecha	Marzo 2026

1. Introducción y Objetivos

Las pruebas unitarias permiten verificar el correcto funcionamiento de unidades individuales de código de forma aislada. En esta práctica se aplican tests unitarios con

JUnit 5 integrado en NetBeans IDE, sobre dos proyectos Java: una Calculadora y un Conversor de Temperatura.

Objetivos

- Crear proyectos Java con estructura Maven en NetBeans.
- Implementar clases con métodos básicos.
- Diseñar y ejecutar tests unitarios con JUnit 5.
- Verificar que todos los tests pasan correctamente (barra verde).
- Comprobar el manejo de excepciones con
- Comprobar el manejo de excepciones mediante **assertThrows**

2. Proyecto 1: Calculadora

2.1 Creación del proyecto en NetBeans

Abre NetBeans y crea un nuevo proyecto Maven: File → New Project → Maven → Java Application. Ponle de nombre

Calculadora y como Group ID: `com.testing`. Finaliza el asistente.

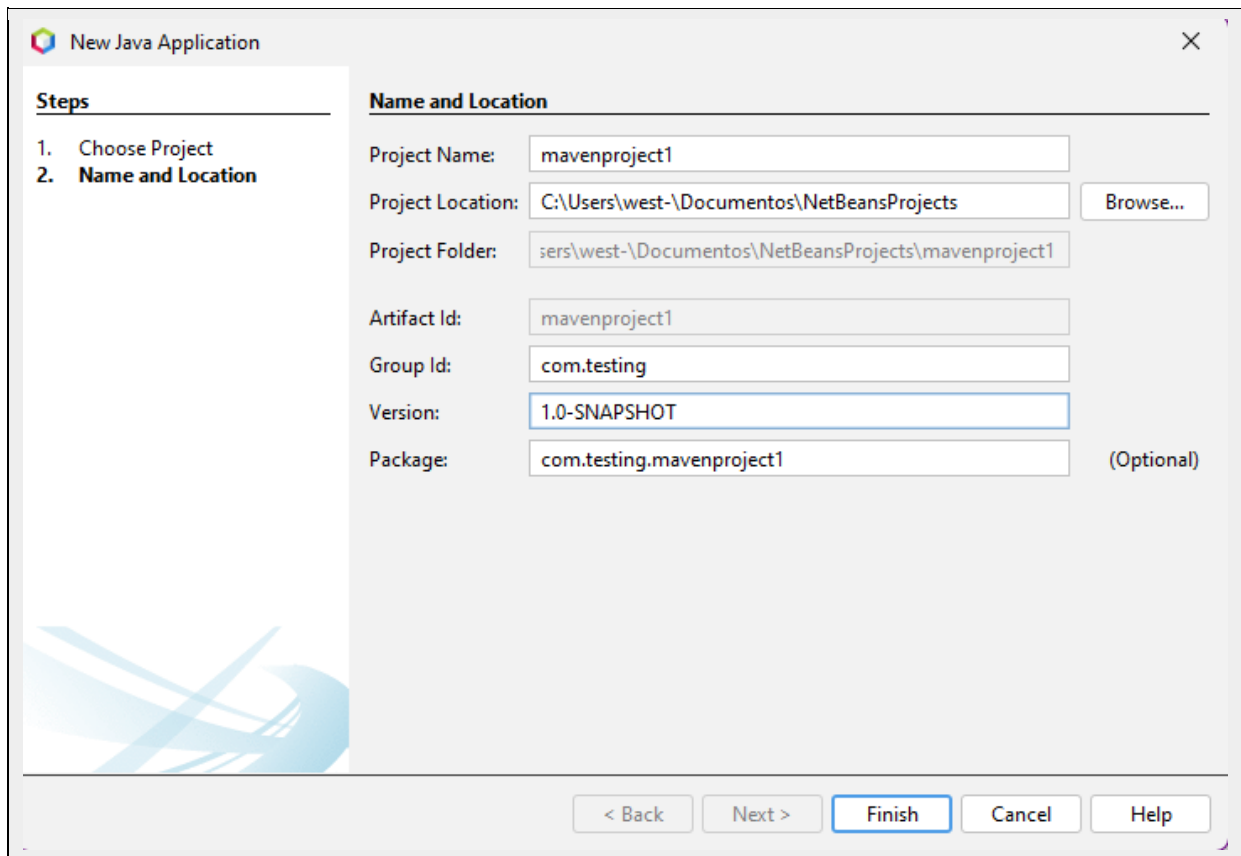


Figura 1 – New Project en NetBeans: proyecto Calculadora

2.2 Clase principal: Testcalculadora.java

Dentro del paquete `com.testing.testcalculadora`, crea la clase `Testcalculadora` con los cuatro métodos aritméticos. El método `dividir` lanza `IllegalArgumentException` si el divisor es cero.

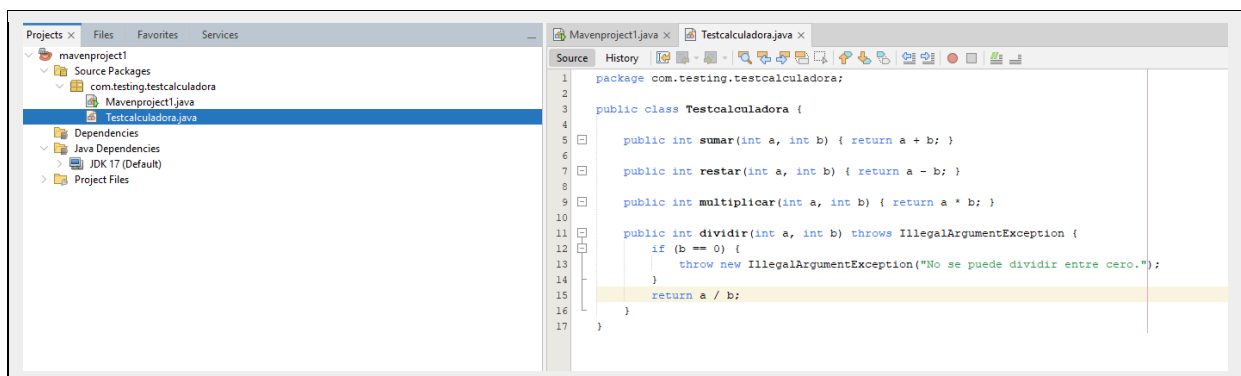


Figura 2 – Testcalculadora.java en el editor de NetBeans

2.3 Añadir dependencia JUnit 5 al pom.xml

Para que JUnit 5 esté disponible, añade la siguiente dependencia en el pom.xml del proyecto (dentro de la etiqueta <dependencies>):

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>
</dependency>
```

Código 2 – Dependencia JUnit 5 en pom.xml



Figura 3 – pom.xml del proyecto Calculadora con JUnit 5

2.4 Clase de test: Calculadortestpruebas.java

En la carpeta Test Packages, crea la clase Calculadortestpruebas con los 5 tests. Incluye un test específico para verificar la excepción de división por cero con assertThrows:

```
import com.testing.testcalculadora.Testcalculadora;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Calculadortestpruebas {

    private final Testcalculadora calculadora = new Testcalculadora();

    @Test
    public void testSumar() {
        assertEquals(5, calculadora.sumar(2, 3), "La suma de 2 y 3 deberia ser 5.");
    }

    @Test
    public void testRestar() {
        assertEquals(1, calculadora.restar(3, 2), "La resta de 3 y 2 deberia ser 1.");
    }

    @Test
    public void testMultiplicar() {
```

```

        assertEquals(6, calculadora.multiplicar(2, 3), "La multiplicacion deberia ser 6.");
    }

    @Test
    public void testDividir() {
        assertEquals(2, calculadora.dividir(6, 3), "La division de 6 entre 3 deberia ser 2.");
    }

    @Test
    public void testDividirPorCero() {
        Exception ex = assertThrows(IllegalArgumentException.class, () -> {
            calculadora.dividir(6, 0);
        });
        assertEquals("No se puede dividir entre cero.", ex.getMessage());
    }
}

```

Código 3 – Calculadoratestpruebas.java

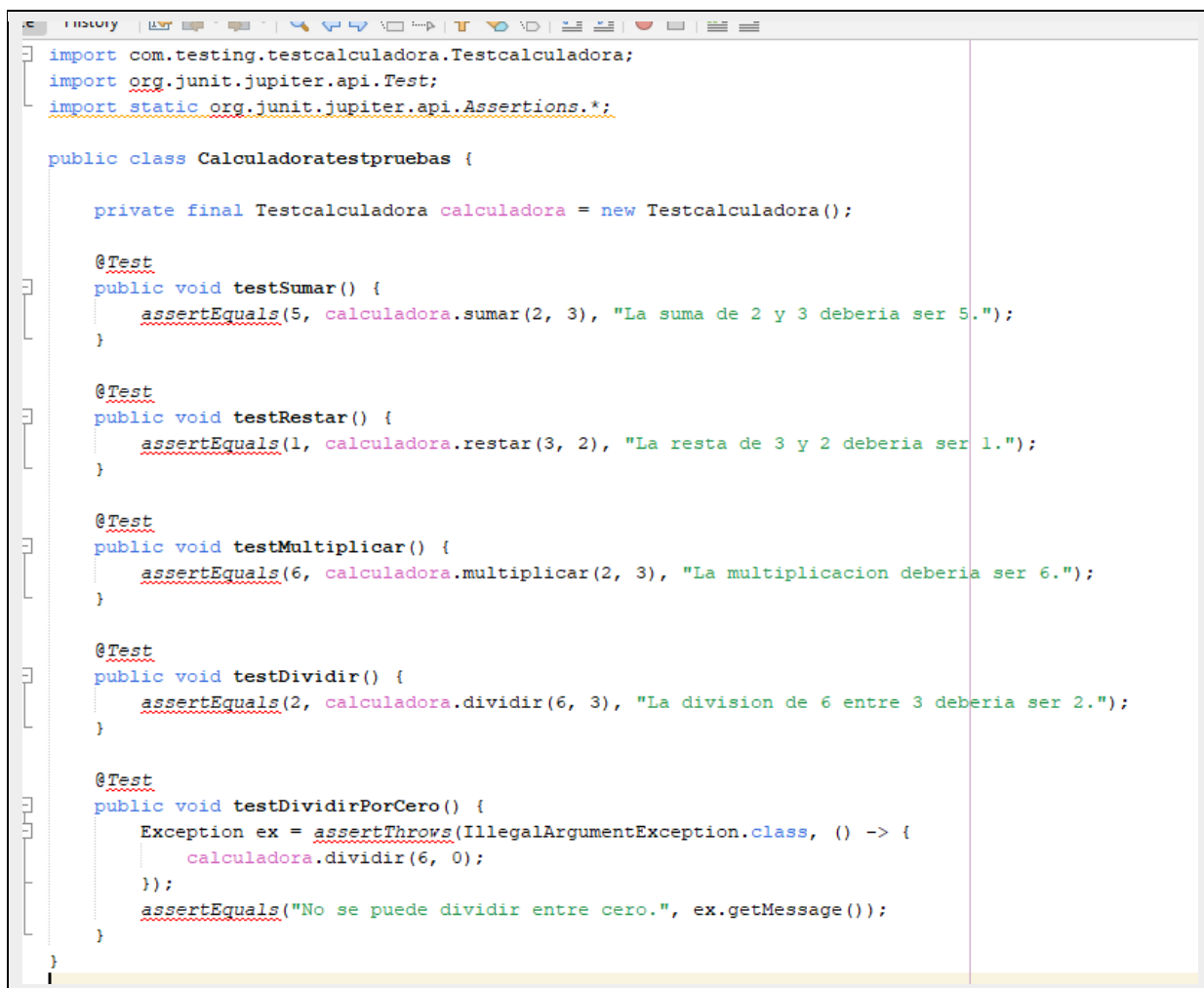


Figura 4 – Calculadoratestpruebas.java en el editor de NetBeans

2.5 Ejecución de los tests

Para ejecutar los tests: clic derecho sobre el proyecto Calculadora en el árbol de proyectos → Test. NetBeans ejecutará todos los métodos anotados con `@Test` y mostrará los resultados.

```

RESULTADO ESPERADO - Proyecto Calculadora
-----
Tests run: 5    |   Failures: 0    |   Errors: 0    |   Skipped: 0
testSumar ..... PASSED
testRestar ..... PASSED
testMultiplicar ..... PASSED
testDividir ..... PASSED
testDividirPorCero .... PASSED
-----
BUILD SUCCESS

```

Resultado esperado – todos los tests en verde

```

Input - Test (CalculadoraTest) x
> Nothing to compile - all classes are up to date.
> --- resources:3.3.1:testResources (default-testResources) @ mavenproject1 ---
> skip non existing resourceDirectory C:\Users\west-\Documentos\NetBeansProjects\mavenproject1\src\test\resources
> --- compiler:3.13.0:testCompile (default-testCompile) @ mavenproject1 ---
> Recompiling the module because of changed source code.
> Compiling 1 source file with javac [debug release 17] to target\test-classes
> --- surefire:3.2.5:test (default-cli) @ mavenproject1 ---
> Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider

-----
T E S T S
-----
Running Test.CalculadoraTest
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.027 s -- in Test.CalculadoraTest

Results:

Tests run: 5, Failures: 0, Errors: 0, Skipped: 0

-----
BUILD SUCCESS
-----
Total time: 1.741 s
Finished at: 2026-03-13T23:22:56Z
-----

```

Figura 5 – Tests del proyecto Calculadora: 5/5 PASSED

2.6 Resumen de tests de la Calculadora

Test	Entrada	Resultado esperado	Aserción
testSumar()	sumar(2, 3)	5	assertEquals
testRestar()	restar(3, 2)	1	assertEquals
testMultiplicar()	multiplicar(2, 3)	6	assertEquals
testDividir()	dividir(6, 3)	2	assertEquals
testDividirPorCero()	dividir(6, 0)	IllegalArgumentException	assertThrows

Tabla 1 – Tests del proyecto Calculadora

3. Proyecto 2: Conversor de Temperatura

3.1 Creación del proyecto en NetBeans

Crea un segundo proyecto Maven en NetBeans: File → New Project → Maven → Java Application. Nombre: ConversorTemperatura, Group ID: com.testing.

[CAPTURA AQUÍ]

Captura 6 – Ventana de creación del proyecto ConversorTemperatura

Captura la ventana New Project de NetBeans mostrando el nombre ConversorTemperatura, el Group ID com.testing y el Package.

Figura 6 – New Project en NetBeans: proyecto ConversorTemperatura

3.2 Clase principal: Conversortemperatura.java

Dentro del paquete com.testing.conversortemperatura, crea la clase Conversortemperatura con los dos métodos de conversión:

```
package com.testing.conversortemperatura;

public class Conversortemperatura {

    // Convierte de Celsius a Fahrenheit
    public double celsiusAFahrenheit(double celsius) {
        return (celsius * 9 / 5) + 32;
    }

    // Convierte de Fahrenheit a Celsius
    public double fahrenheitACelsius(double fahrenheit) {
        return (fahrenheit - 32) * 5 / 9;
    }
}
```

Código 4 – Conversortemperatura.java

Name and Location

Project Name:

Project Location:

Project Folder:

Artifact Id:

Group Id:

Version:

Package: (Optional)

Figura 7 – Conversortemperatura.java en el editor de NetBeans

3.3 Clase de test: ConversorTemperaturaTest.java

En Test Packages, crea la clase ConversorTemperaturaTest con 2 métodos de test. Se usa un delta de 0.001 para la comparación de valores decimales con assertEquals:

```
package ConversorTemperaturaTest;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
import com.testing.conversortemperatura.*;

public class ConversorTemperaturaTest {

    private final Conversortemperatura conversor = new Conversortemperatura();

    @Test
    public void testCelsiusAFahrenheit() {
        assertEquals(32.0, conversor.celsiusAFahrenheit(0), 0.001, "0 C = 32 F");
        assertEquals(212.0, conversor.celsiusAFahrenheit(100), 0.001, "100 C = 212 F");
    }

    @Test
    public void testFahrenheitACelsius() {
        assertEquals(0.0, conversor.fahrenheitACelsius(32), 0.001, "32 F = 0 C");
        assertEquals(100.0, conversor.fahrenheitACelsius(212), 0.001, "212 F = 100 C");
    }
}
```

Código 5 – ConversorTemperaturaTest.java

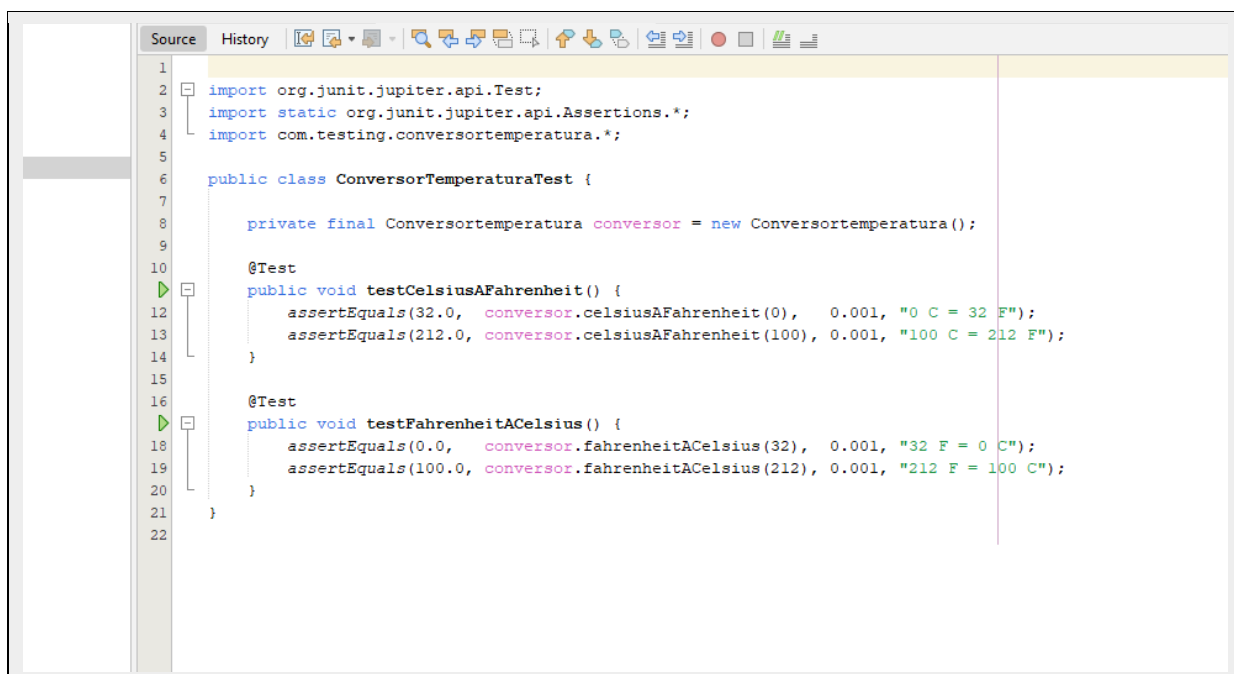


Figura 8 – ConversorTemperaturaTest.java en el editor de NetBeans

3.4 Ejecución de los tests

Clic derecho sobre el proyecto ConversorTemperatura → Test. Los 2 tests deben pasar en verde.

RESULTADO ESPERADO – Proyecto ConversorTemperatura

```
-----
Tests run: 2    |   Failures: 0    |   Errors: 0    |   Skipped: 0
testCelsiusAFahrenheit .... PASSED
testFahrenheitACelsius .... PASSED
-----
```

BUILD SUCCESS

Resultado esperado – todos los tests en verde

T E S T S

Running ConversorTemperaturaTest

Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.030 s -- in C

Results:

Tests run: 2, Failures: 0, Errors: 0, Skipped: 0

BUILD SUCCESS

Total time: 1.299 s

Finished at: 2026-03-13T23:38:06Z

Captura la ventana de resultados de JUnit en NetBeans mostrando los 2 tests en verde (PASSED) y el mensaje BUILD SUCCESS en la pestaña Output.

Figura 9 – Tests del proyecto ConversorTemperatura: 2/2 PASSED

3.5 Resumen de tests del Conversor

Test	Caso 1	Caso 2	Delta
testCelsiusAFahrenheit()	0°C → 32°F	100°C → 212°F	0.001
testFahrenheitACelsius()	32°F → 0°C	212°F → 100°C	0.001

Tabla 2 – Tests del proyecto ConversorTemperatura

4. Estructura de los Proyectos en NetBeans

Ambos proyectos siguen la estructura estándar Maven. Las clases de producción se ubican en Source Packages y las clases de test en Test Packages:

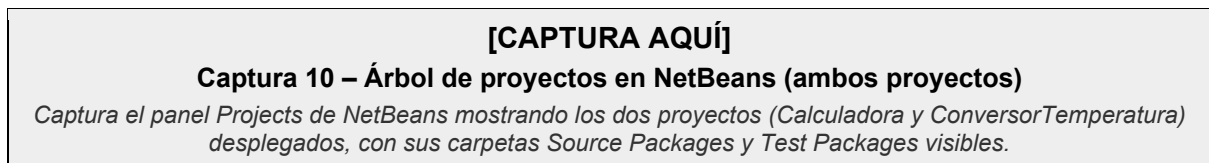


Figura 10 – Estructura de ambos proyectos en el panel Projects de NetBeans

Proyecto	Paquete	Clase Principal	Clase de Test
Calculadora	com.testing.testcalculadora	Testcalculadora.java	Calculadortestpruebas.java
ConversorTemperatura	com.testing.conversortemperatura	Conversortemperatura.java	ConversorTemperaturaTest.java

Tabla 3 – Estructura de los proyectos

5. Conceptos Clave Aplicados

Elemento	Descripción
@Test	Anotación de JUnit 5 que marca un método como caso de prueba unitaria.
assertEquals(exp, act, delta, msg)	Verifica que el valor obtenido coincide con el esperado. Delta es la tolerancia para doubles.
assertThrows(Class, executable)	Verifica que la ejecución de un bloque de código lanza la excepción especificada.
IllegalArgumentException	Excepción lanzada por dividir() cuando el divisor es cero. Capturada y verificada en el test.
Delta / tolerancia (0.001)	Margen en comparaciones de doubles para evitar errores de precisión de punto flotante.

Tabla 4 – Conceptos clave de JUnit 5 utilizados en la práctica

6. Conclusiones

Esta práctica ha permitido comprender y aplicar los fundamentos del testing unitario en Java con JUnit 5 y NetBeans IDE. Se han desarrollado con éxito dos proyectos Maven con sus respectivas clases de producción y de test:

- **Calculadora**El proyecto
- **Calculadora**: 5 tests que cubren las cuatro operaciones básicas y la excepción por división entre cero.
- **Conversor de Temperatura**: 2 tests que verifican conversiones en ambas direcciones usando tolerancia de punto flotante.
- Todos los tests han resultado **PASSED (barra verde)**, confirmando la corrección del código.
- Se han practicado las aserciones más habituales: **`assertEquals`** y **`assertThrows`**.

Las pruebas unitarias son una herramienta esencial para garantizar la calidad del software, detectar errores de forma temprana y facilitar el mantenimiento del código a lo largo del tiempo.

Fin de la memoria – UT3 Práctica A9.1